

Tab này sử dụng thư viện Bitcraze với data realtime nhận từ PMW390

arduino

```
// =====
// PMW3901 Frame Grab — ESP32-S3 Supermini
// Init:      flow.begin() từ Bitcraze library (đã xác nhận hoạt động)
// Frame Grab: Tự implement dựa trên source thật của library
// Fix chính: Thêm timeout vào mọi do-while để tránh treo vô tận
// Timing:    Lấy từ source thật: 50µs tSRAD, 200µs sau CS HIGH
// =====

#include <SPI.h>
#include <Bitcraze_PMW3901.h>

#define FRAME_WIDTH  35
#define FRAME_HEIGHT 35
#define TOTAL_PIXELS (FRAME_WIDTH * FRAME_HEIGHT)

#define PMW_CS_PIN  4
#define PMW_SCK_PIN 7
#define PMW_MOSI_PIN 5
#define PMW_MISO_PIN 6

Bitcraze_PMW3901 flow(PMW_CS_PIN);
char frameBuffer[TOTAL_PIXELS];

// =====
// SPI LOW-LEVEL — timing lấy từ source thật Bitcraze library
// registerWrite: 50µs trước/sau mỗi byte, 200µs sau CS HIGH
// registerRead: 50µs trước data, 100µs sau data, 200µs sau CS HIGH
// =====

void regWrite(uint8_t reg, uint8_t value) {
  SPI.beginTransaction(SPISettings(4000000, MSBFIRST, SPI_MODE3));
  digitalWrite(PMW_CS_PIN, LOW);
  delayMicroseconds(50);
  SPI.transfer(reg | 0x80);
  SPI.transfer(value);
  delayMicroseconds(50);
  digitalWrite(PMW_CS_PIN, HIGH);
  SPI.endTransaction();
  delayMicroseconds(200);
}

uint8_t regRead(uint8_t reg) {
  SPI.beginTransaction(SPISettings(4000000, MSBFIRST, SPI_MODE3));
  digitalWrite(PMW_CS_PIN, LOW);
  delayMicroseconds(50);
  SPI.transfer(reg & 0x7F);
  delayMicroseconds(50);      // tSRAD từ source thật
```

```

uint8_t value = SPI.transfer(0);
delayMicroseconds(100);
digitalWrite(PMW_CS_PIN, HIGH);
SPI.endTransaction();
delayMicroseconds(200);
return value;
}

// =====
// ENABLE FRAME BUFFER — với timeout, không treo vô tận
// Source: Bitcraze_PMW3901.cpp::enableFrameBuffer()
// =====

bool enableFrameBuffer() {
    regWrite(0x7F, 0x07);
    regWrite(0x41, 0x1D);
    regWrite(0x4C, 0x00);
    regWrite(0x7F, 0x08);
    regWrite(0x6A, 0x38);
    regWrite(0x7F, 0x00);
    regWrite(0x55, 0x04);
    regWrite(0x40, 0x80);
    regWrite(0x4D, 0x11);
    regWrite(0x70, 0x00);
    regWrite(0x58, 0xFF); // Trigger: bắt đầu capture đầu tiên

    // Chờ chip báo sẵn sàng — có timeout 5000 lần thử
    int timeout = 5000;
    uint8_t temp;
    uint8_t check;
    do {
        temp = regRead(0x58);
        check = temp >> 6;
        timeout--;
    } while (check == 0x03 && timeout > 0);

    if (timeout <= 0) {
        Serial.println("FAIL: enableFrameBuffer timeout!");
        return false;
    }

    delayMicroseconds(50);
    Serial.println("OK: Frame Buffer enabled");
    return true;
}

// =====
// READ 1 FRAME — với timeout trên mọi vòng lặp
// Source: Bitcraze_PMW3901.cpp::readFrameBuffer()
//
// Encoding của register 0x58:
// bits[7:6] = 00 hoặc 11 → invalid, đọc lại

```

```

// bits[7:6] = 01 → byte này chứa 6 bits trên của pixel (bits[5:0])
// bits[7:6] = 10 → byte này chứa 2 bits dưới của pixel (bits[3:2])
//
// Với mỗi pixel: đọc byte "01" trước → shift left 2 → đọc byte "10" tiếp
// → lấy bits[3:2] → ghép thành pixel 8-bit hoàn chỉnh
// =====

bool readFrame(char* buf) {
    int count = 0;
    uint8_t a, b, hold, pixel;
    uint8_t mask = 0x0C; // bits[3:2]

    for (int ii = 0; ii < TOTAL_PIXELS; ii++) {
        // Đọc cho đến khi có byte hợp lệ (01 hoặc 10)
        int timeout = 2000;
        do {
            a = regRead(0x58);
            hold = a >> 6;
            timeout--;
        } while ((hold == 0x03 || hold == 0x00) && timeout > 0);

        if (timeout <= 0) {
            Serial.print("Timeout tại pixel "); Serial.println(ii);
            return false;
        }

        if (hold == 0x01) {
            // Byte "01": 6 bits trên nằm ở bits[5:0] của a
            b = regRead(0x58); // Byte "10" tiếp theo: 2 bits dưới ở bits[3:2]
            pixel = a << 2; // Đẩy 6 bits lên vị trí [7:2]
            pixel += (b & mask); // Gắn 2 bits dưới vào vị trí [3:2]
            buf[count++] = pixel;
        }
        // Nếu hold == 0x02 (byte "10" xuất hiện không đúng lượt): bỏ qua
    }

    // Re-arm: kích hoạt capture tiếp theo
    regWrite(0x70, 0x00);
    regWrite(0x58, 0xFF);

    int timeout = 5000;
    uint8_t temp, check;
    do {
        temp = regRead(0x58);
        check = temp >> 6;
        timeout--;
    } while (check == 0x03 && timeout > 0);

    if (count < TOTAL_PIXELS) {
        Serial.print("Warning: chỉ fill được "); Serial.print(count);
        Serial.println("/1225 pixels");
    }

    return (count > 0);
}

```

```

}

// =====
// SETUP
// =====

void setup() {
  Serial.begin(115200);
  delay(3000);

  pinMode(PMW_CS_PIN, OUTPUT);
  digitalWrite(PMW_CS_PIN, HIGH);

  // Khởi tạo SPI với custom pins TRƯỚC khi gọi flow.begin()
  SPI.begin(PMW_SCK_PIN, PMW_MISO_PIN, PMW_MOSI_PIN, PMW_CS_PIN);

  Serial.println("=== PMW3901 Init ===");

  if (!flow.begin()) {
    Serial.println("FAIL: flow.begin() that bai. Kiem tra day noi.");
    while (1);
  }
  Serial.println("OK: flow.begin() thanh cong");

  if (!enableFrameBuffer()) {
    Serial.println("FAIL: Khong the vao Frame Grab Mode.");
    while (1);
  }

  Serial.println(">>> Bat dau stream...");
}

// =====
// LOOP
// =====

void loop() {
  if (readFrame(frameBuffer)) {
    Serial.write('*');
    Serial.write((uint8_t*)frameBuffer, TOTAL_PIXELS);
  } else {
    Serial.println("Frame read failed, retrying...");
    delay(100);
  }
}

```

processing(nhớ chỉnh COMX)

```
import processing.serial.*;

Serial myPort;
int frameWidth = 35;
int frameHeight = 35;
int scaleFactor = 15; // Phóng to mỗi pixel lên 15 lần cho dễ nhìn (kích thước 525x525)

byte[] frameBuffer = new byte[1225];
int byteCount = 0;
boolean isReadingFrame = false;

void setup() {
    size(525, 525);
    noStroke(); // Tắt viền đen của các điểm ảnh

    // Mở cổng COM18 giao tiếp với ESP32
    myPort = new Serial(this, "COM18", 115200);
}

void draw() {
    // Liên tục hút dữ liệu từ cổng COM18
    while (myPort.available() > 0) {
        int inByte = myPort.read();

        if (!isReadingFrame) {
            // Đang ở trạng thái 1: TÌM CỜ BÁO HIỆU
            if (inByte == '*') {
                isReadingFrame = true; // Tìm thấy cờ -> Chuyển trạng thái
                byteCount = 0;        // Reset bộ đếm để chuẩn bị hứng 1225 byte
            }
        } else {
            // Đang ở trạng thái 2: HỨNG DỮ LIỆU TỪNG PIXEL
            frameBuffer[byteCount] = (byte)inByte;
            byteCount++;

            // Nếu đã hứng đủ 1225 pixel
            if (byteCount == frameWidth * frameHeight) {
                drawFrame(); // Gọi hàm vẽ ngay lập tức!
                isReadingFrame = false; // Reset lại trạng thái tìm cờ '*' cho khung hình tiếp theo
            }
        }
    }
}

// Hàm thực hiện việc vẽ ảnh ra màn hình
void drawFrame() {
    for (int y = 0; y < frameHeight; y++) {
        for (int x = 0; x < frameWidth; x++) {
            int i = y * frameWidth + x;
```

```
int pixelValue = frameBuffer[i] & 0xff; // Chuyển byte có dấu thành số nguyên 0-255  
  
fill(pixelValue); // Đặt màu xám tương ứng với độ sáng của pixel  
rect(x * scaleFactor, y * scaleFactor, scaleFactor, scaleFactor); // Vẽ hình vuông  
}  
}  
}
```



